

Arquitecturas para Ciencia de Datos

Tema 8

Prof. Ing. Angel Brito Segura

1. Ciencia de Datos

La ciencia de datos es un campo interdisciplinario que utiliza métodos, procesos, algoritmos y sistemas para extraer conocimiento e *insights* de datos estructurados y no estructurados. Combina técnicas de estadística, análisis de datos, aprendizaje automático (ML) y visualización para interpretar y comunicar los resultados obtenidos a partir de grandes conjuntos de datos.

2. Arquitectura de Datos

La arquitectura de datos se refiere a la estructura y organización de los datos dentro de una empresa o sistema. Incluye la forma en que los datos se recopilan, gestionan, procesan, acceden y distribuyen sus datos. Una buena arquitectura de datos es crucial para garantizar la eficiencia, escalabilidad y seguridad en el manejo de grandes volúmenes de datos, especialmente en el contexto de la ciencia de datos. Una arquitectura de datos bien diseñada facilita la integración de diferentes fuentes de datos, mejora la calidad de los datos y permite un análisis más efectivo, lo que a su vez puede conducir a mejores decisiones empresariales y resultados más precisos. En las organizaciones tradicionales, se utilizaban esquemas monolíticos (Data Warehouses) con un alto costo de recursos. La arquitectura moderna resuelve esto desvinculando por completo el almacenamiento del procesamiento. Esto se logra mediante el uso de *Data Lakes*, que permiten almacenar grandes volúmenes de datos en su formato original, y herramientas de procesamiento como *Apache Spark*, que pueden escalar horizontalmente para manejar grandes cargas de trabajo de procesamiento de datos.

2.1. Arquitectura moderna de datos

La arquitectura moderna de datos se basa en la separación de almacenamiento y procesamiento, lo que permite a las organizaciones manejar grandes cantidades de datos sin incurrir en los altos costos asociados con los esquemas monolíticos tradicionales, facilitando la integración de diferentes fuentes de datos y mejorando la calidad y el análisis de los datos:

- **Capa de Almacenamiento (Data Lakes):** Repositorios de almacenamiento de objetos (como Amazon S3 o Google Cloud Storage) diseñados para soportar datos estructurados y no estructurados de forma económica. Cumplen el rol de un servidor de archivos global y distribuido.
- **Capa de Procesamiento:** Herramientas que extraen los datos almacenados para transformarlos y analizarlos. Aquí actúan tecnologías como Apache Hadoop y Apache Spark, funcionando como servidores de aplicaciones de alto rendimiento.

3. Servicio administrado para Apache Spark en Google Cloud

En este laboratorio en Google Skills, se aprenderá a usar este servicio para procesar grandes volúmenes de datos de manera eficiente y escalable. Se cubrirán aspectos como la creación de clústeres de Spark, la ejecución de trabajos de procesamiento de datos y la integración con otros servicios para análisis avanzados. Ingresa al laboratorio de costo Managed Service for Apache Spark: Qwik Start – Command Line que se encuentra en la siguiente dirección:

https://www.skills.google/focuses/585?catalog_rank=%7B%22rank%22%3A1%2C%22num_filters%22%3A0%2C%22has_search%22%3Atrue%7D&parent=catalog&search_id=83334065

NOTA 1: Recuerda que al ser de paga, **SOLO SE PUEDE REALIZAR UNA VEZ** el laboratorio.

NOTA 2: L@s alumn@s que entreguen como **Tarea 4** capturas de pantalla de las verificaciones correctas en cada actividad solicitada del laboratorio, la aproximación del valor de π dada por el servicio y la modificación correcta de los nodos trabajadores del clúster (6 capturas de pantalla) se tendrá un punto extra sobre diez del promedio obtenido en los **Ejercicios Prácticos**

Anteriormente conocido como **Cloud Dataproc** (para implementaciones en **Compute Engine**) y **Google Cloud Serverless for Apache Spark**, el **Managed Service for Apache Spark** es un servicio administrado que permite ejecutar clústeres de Apache Spark y Apache Hadoop de forma rápida, sencilla y rentable.

En la administración de infraestructura tradicional, configurar un clúster de este tipo requiere días de instalación de dependencias, configuración de redes y sincronización de nodos. Este servicio abstrae esa complejidad, permitiendo desplegar la arquitectura en minutos y apagarla cuando el procesamiento finaliza, optimizando el consumo de recursos. En este ecosistema distribuido interactuarán con dos conceptos de red:

- **Master Node:** Servidor central que coordina, administra los recursos y distribuye las tareas.
- **Worker Nodes:** Servidores que ejecutan el procesamiento de forma paralela (la escalabilidad se aplica agregando o eliminando estos nodos dinámicamente según la demanda).

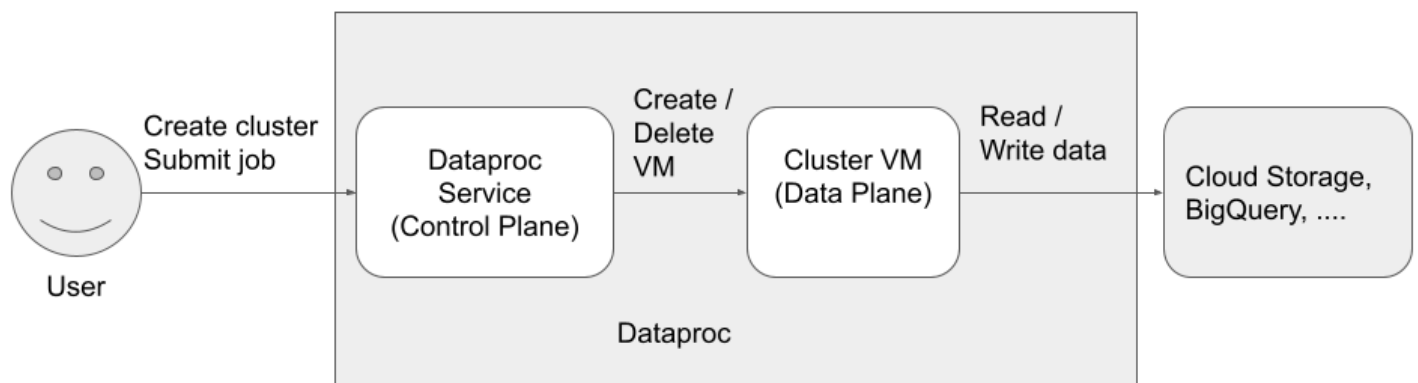


Figura 1: Servicio administrado para Apache Spark en Google Cloud

3.1. Ecosistema Hadoop

Hadoop es un framework de código abierto diseñado para almacenar y procesar grandes volúmenes de datos (*Big Data*) distribuyéndolos a través de clústeres de computadoras convencionales. El ecosistema Hadoop incluye una variedad de herramientas y tecnologías que trabajan juntas para facilitar el almacenamiento, procesamiento y análisis de datos a gran escala:

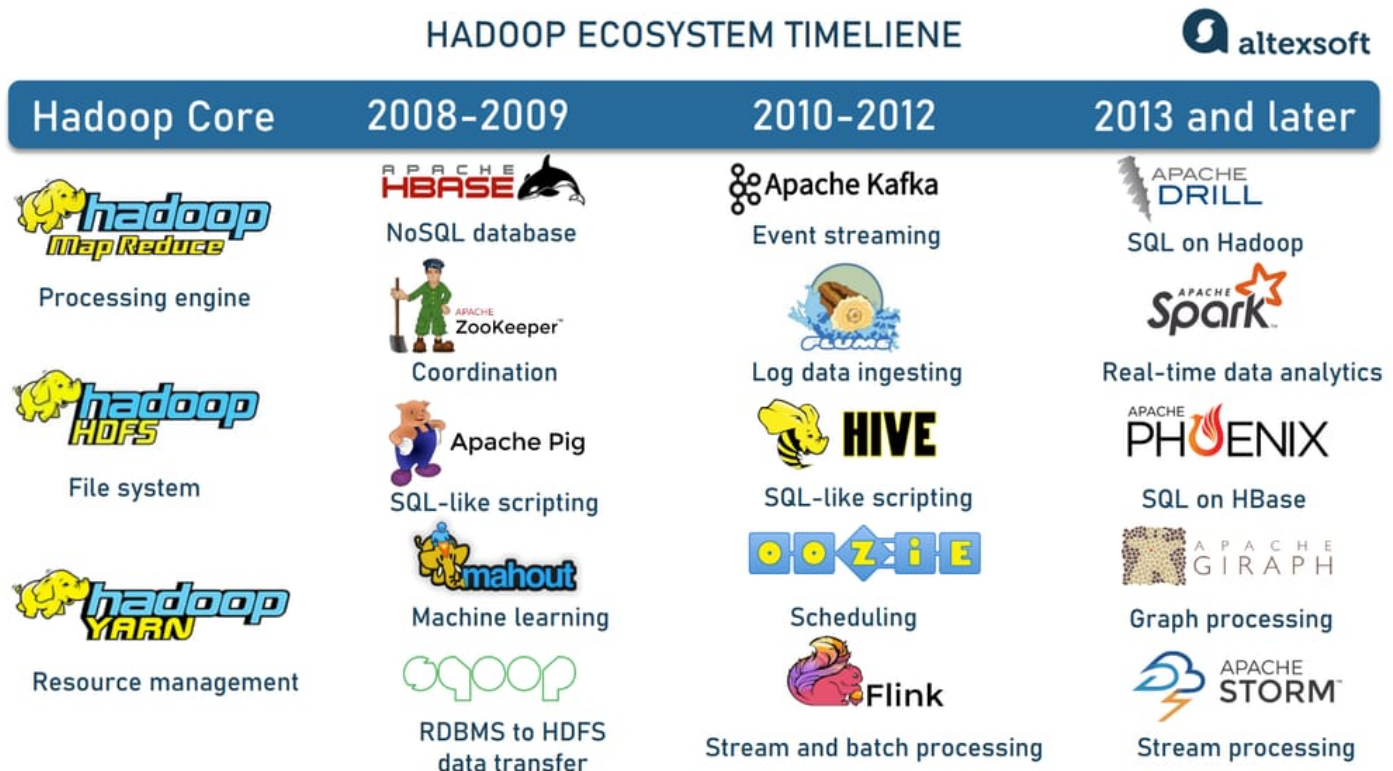


Figura 2: Evolución del ecosistema Hadoop

- **HDFS (Hadoop Distributed File System)**: Sistema de archivos distribuido que permite almacenar grandes volúmenes de datos en un clúster de computadoras.
- **MapReduce**: Modelo de programación para procesar grandes conjuntos de datos en paralelo a través de un clúster.
- **YARN (Yet Another Resource Negotiator)**: Sistema de gestión de recursos que permite ejecutar múltiples aplicaciones en un clúster Hadoop.
- **Hive**: Data warehouse que facilita la consulta y análisis de datos almacenados en HDFS utilizando un lenguaje similar a SQL.
- **Pig**: Plataforma para crear programas MapReduce utilizando un lenguaje de alto nivel llamado Pig Latin.
- **HBase**: Base de datos NoSQL distribuida que se ejecuta sobre HDFS, diseñada para manejar grandes cantidades de datos no estructurados.

- **Spark:** Motor de procesamiento rápido y general para big data, que puede ejecutar tareas MapReduce más rápidamente que Hadoop.

Dentro de sus ventajas se encuentra la alta escalabilidad horizontal, tolerancia a fallos inherente, y rentabilidad al usar hardware de consumo estándar. Sin embargo, también presenta desafíos como el rendimiento variable dependiendo del tipo de carga de trabajo, en la que se puede encontrar una alta latencia (no es ideal para procesamiento en tiempo real), ineficiencia con archivos pequeños y una curva de aprendizaje pronunciada para su administración en entornos on-premise.

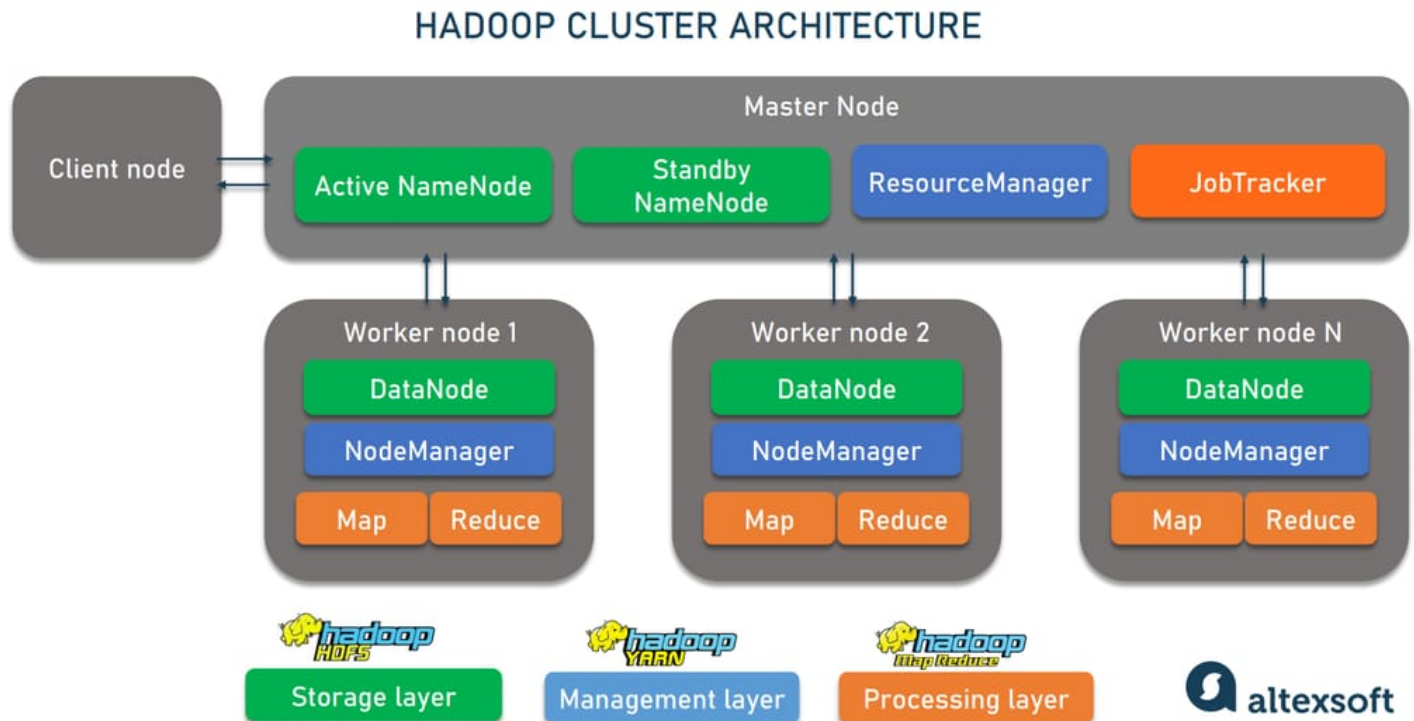


Figura 3: Arquitectura de un clúster de Hadoop

3.1.1. Arquitectura de Almacenamiento

El **Hadoop Distributed File System (HDFS)** es el corazón de Hadoop. Funciona bajo una arquitectura Maestro/Esclavo. El NameNode (Maestro) gestiona el sistema de archivos y el control de acceso, mientras que los DataNodes (Esclavos) almacenan los bloques de datos reales. HDFS está diseñado para ser altamente tolerante a fallos, replicando la información en múltiples nodos.

Sin embargo, esta arquitectura puede presentar un punto único de falla en el NameNode, aunque existen soluciones como el NameNode secundario para mitigar este riesgo. HDFS es ideal para almacenar grandes volúmenes de datos, pero no es eficiente para archivos pequeños debido a la sobrecarga de metadatos y la latencia asociada con la lectura y escritura de estos archivos.

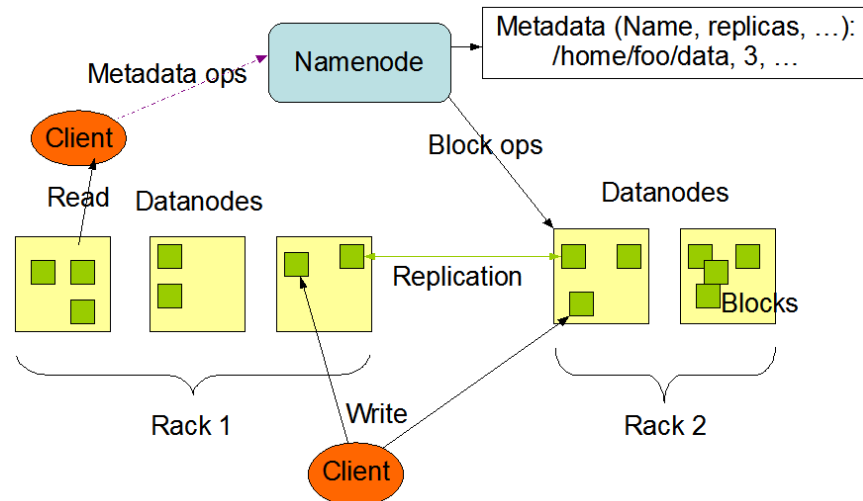


Figura 4: Arquitectura de almacenamiento en Hadoop (HDFS)

3.1.2. Arquitectura de Procesamiento

El modelo de procesamiento MapReduce de Hadoop se basa en la división de tareas en dos fases: Map y Reduce. En la fase Map, los datos se procesan en paralelo en los nodos de trabajo, generando pares clave-valor intermedios. En la fase Reduce, estos pares se agrupan y procesan para producir el resultado final.

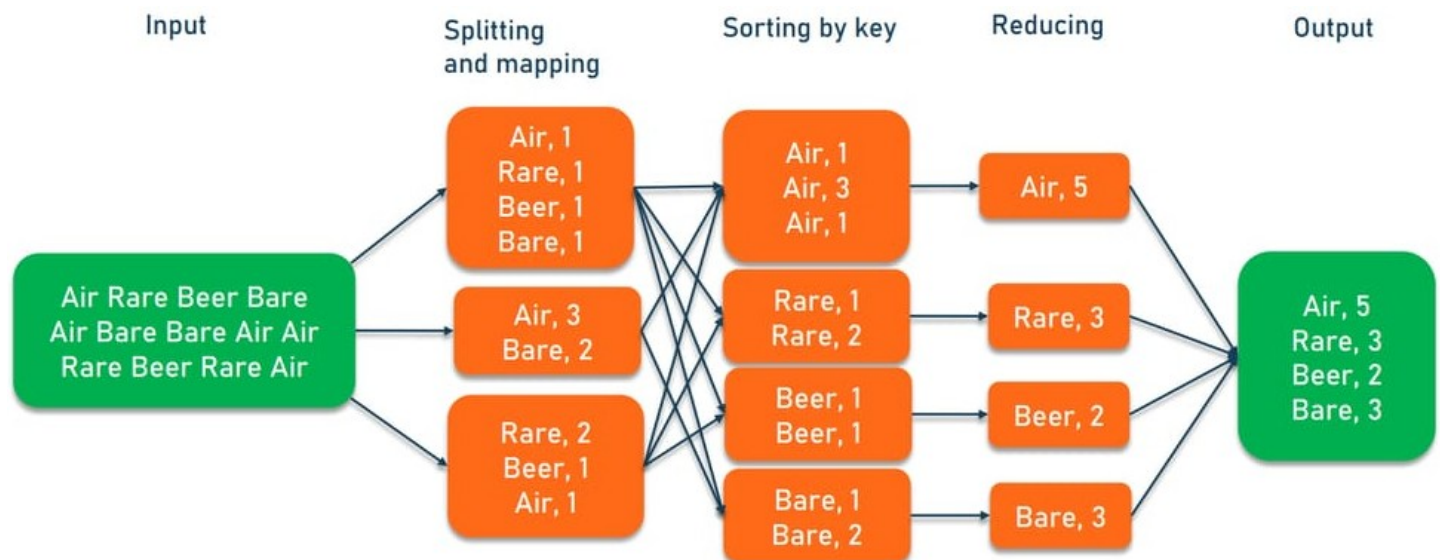


Figura 5: Proceso MapReduce en Hadoop

Este enfoque es eficiente para tareas que pueden ser paralelizadas, pero puede resultar ineficiente para cargas de trabajo que requieren procesamiento en tiempo real o iterativo, debido a la necesidad de escribir y leer datos intermedios en el disco, lo que introduce latencia. Además, la gestión de recursos y la optimización de tareas pueden ser complejas, especialmente en clústeres grandes, lo que puede afectar el rendimiento general del sistema.

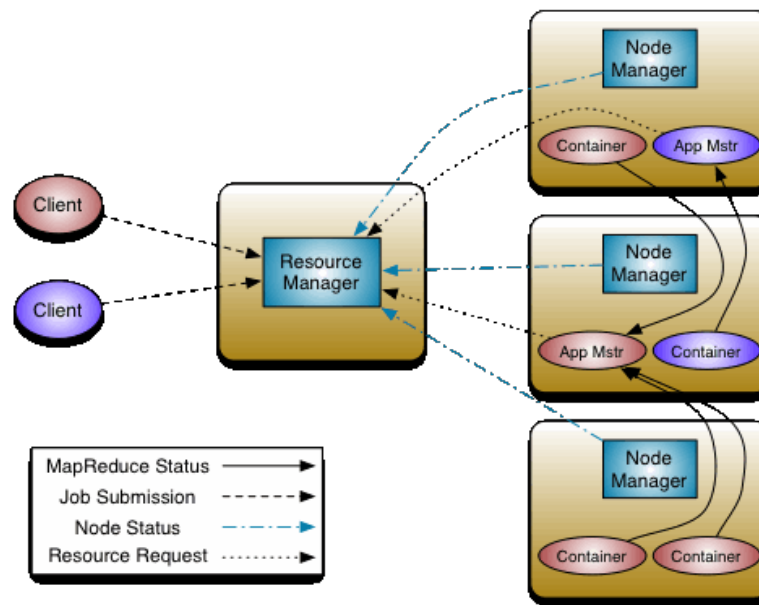


Figura 6: Arquitectura de YARN en Hadoop

3.2. Apache Spark

Apache Spark es un motor de procesamiento de datos de código abierto diseñado para ser rápido y general. A diferencia de Hadoop, Spark procesa los datos en memoria, lo que permite una ejecución mucho más rápida para ciertas cargas de trabajo, especialmente aquellas que requieren procesamiento iterativo o en tiempo real.

Compatible con una amplia variedad de lenguajes de programación, incluyendo Java, Scala, Python y R, lo que lo hace accesible a una amplia gama de desarrolladores y científicos de datos. Además, Spark ofrece una serie de bibliotecas integradas para tareas específicas como el aprendizaje automático (MLlib), el procesamiento de gráficos (GraphX), el análisis estructurado (Spark SQL) y el procesamiento de flujos de datos (Spark Streaming), lo que lo convierte en una plataforma versátil para el análisis de big data.

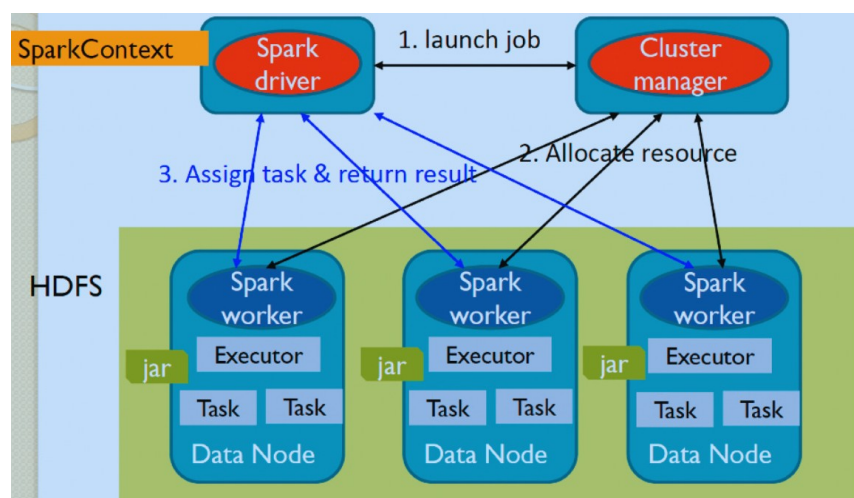


Figura 7: Arquitectura de Apache Spark

Dentro de un clúster de Spark, la arquitectura se basa en un modelo Maestro/Esclavo similar al de Hadoop, pero con una mayor flexibilidad y eficiencia en la gestión de recursos:

- **Driver Program (Nodo Maestro):** Es el cerebro de la aplicación Spark. Mantiene la información del clúster, traduce el código del usuario en tareas ejecutables y planifica su distribución.
- **Cluster Manager:** Administra los recursos físicos (puede ser YARN en Hadoop, Mesos, o el propio de Spark).
- **Worker Nodes / Executors (Nodos Trabajadores):** Son los servidores que ejecutan físicamente las tareas en paralelo y almacenan los datos en memoria caché. Estos nodos se pueden modificar (`-num-workers`) para escalar la capacidad de procesamiento.

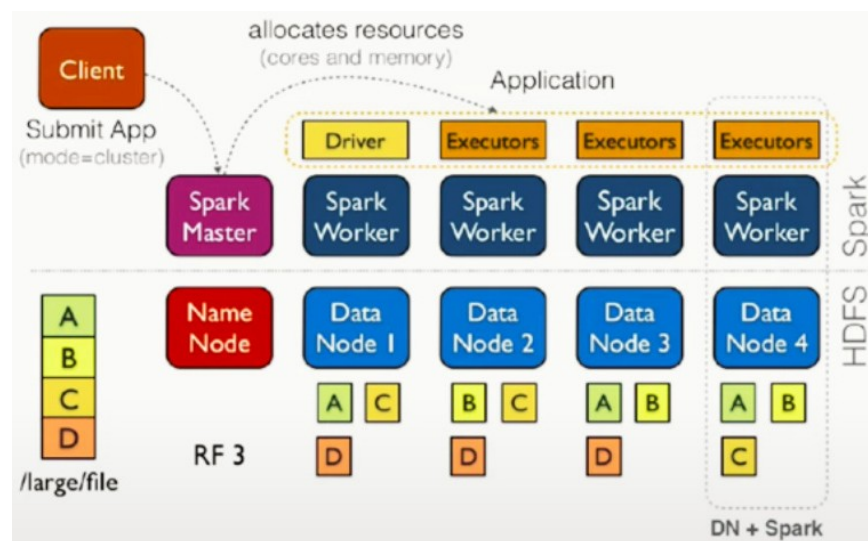


Figura 8: Anatomía de un clúster de Spark

3.3. Ejecución de Spark en un clúster

En un clúster de Spark, el proceso de ejecución se coordina a través de un nodo maestro (Driver) que gestiona la aplicación y distribuye las tareas a los nodos trabajadores (Executors). El Driver es responsable de convertir el código del usuario en tareas que pueden ser ejecutadas en paralelo, mientras que los Executors son responsables de ejecutar estas tareas y devolver los resultados al Driver. Spark utiliza un modelo de programación basado en Resilient Distributed Datasets (RDDs) y DataFrames, lo que permite a los desarrolladores escribir código de alto nivel para procesar grandes conjuntos de datos sin preocuparse por los detalles de la distribución y paralelización.

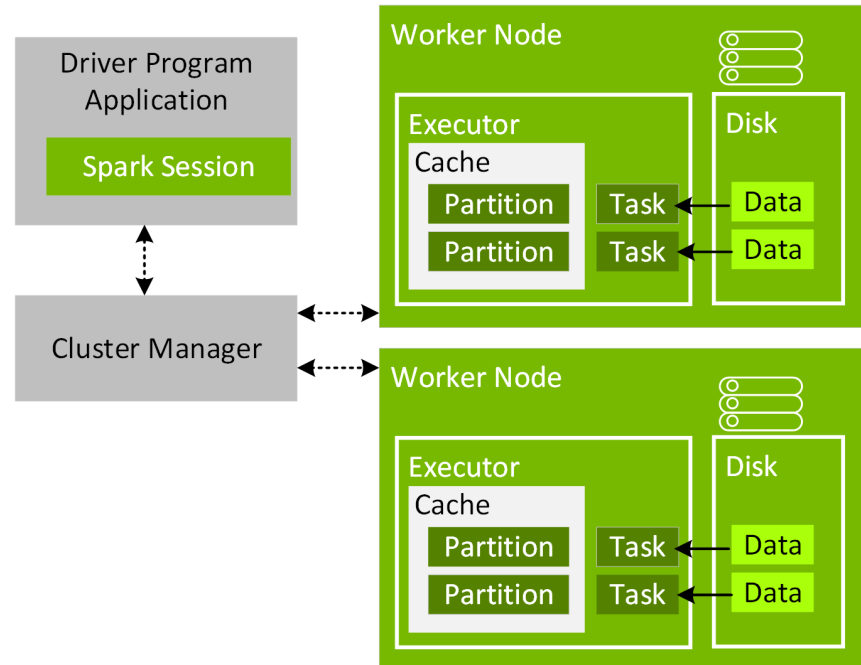


Figura 9: Ejecución de una aplicación Spark en un clúster

4. Alternativas Open Source

Además de Hadoop y Apache Spark, existen otras alternativas de código abierto para el procesamiento de grandes volúmenes de datos, como Apache Flink y Apache Storm. Estas herramientas ofrecen diferentes enfoques y características para el procesamiento de datos en tiempo real, análisis iterativo y manejo de flujos de datos, proporcionando a los usuarios opciones flexibles para abordar sus necesidades específicas de procesamiento de datos.

4.1. Ingesta y Streaming de Datos

Antes de procesar los datos, hay que moverlos desde los servidores web o de aplicaciones hacia el almacenamiento.

- **Apache Kafka:** Es el estándar absoluto de la industria para arquitecturas orientadas a eventos y streaming de datos en tiempo real. Funciona como un sistema de mensajería distribuido de altísimo rendimiento.
- **Apache NiFi:** Originalmente desarrollado por la NSA, es una herramienta visual excelente para automatizar el flujo de datos entre sistemas, ideal para aplicar enrutamiento y transformación al vuelo.

4.2. Almacenamiento y Formatos

La capa equivalente a los HDFS vistos:

- **Amazon S3 / Google Cloud Storage / Azure Blob Storage:** Son los servicios de almacenamiento de objetos más populares en la nube, que ofrecen alta durabilidad, escalabilidad y disponibilidad para almacenar grandes volúmenes de datos.
- **MinIO:** Un servidor de almacenamiento de objetos distribuido y de altísimo rendimiento, totalmente compatible con la API de Amazon S3. Es la herramienta perfecta para que los alumnos practiquen la creación de un Data Lake localmente sin gastar en la nube.
- **Apache Iceberg / Delta Lake (Formatos de Tabla):** Esta es la tendencia más fuerte en la industria. Especialmente útil para conectar los conceptos que los alumnos ven en su laboratorio de bases de datos, ya que llevan la confiabilidad transaccional (propiedades ACID) directamente a los archivos distribuidos del Data Lake, creando lo que se conoce como un Data Lakehouse.

4.3. Procesamiento y Análisis

Si bien Spark es dominante, hay otras herramientas según el tipo de carga de trabajo:

- **Apache Storm:** Un sistema de procesamiento de flujos de datos en tiempo real que permite procesar grandes volúmenes de datos de manera continua, ideal para aplicaciones como análisis de redes sociales, monitoreo de sistemas y detección de fraudes.
- **Apache Flink:** Mientras que Spark es el rey indiscutible del procesamiento por lotes (y micro-lotes), Flink es el líder actual en el procesamiento de datos puramente en tiempo real (streaming continuo), ideal para detección de fraudes o analítica en vivo.
- **Ray:** Un framework open source diseñado para escalar aplicaciones y cargas de trabajo de Inteligencia Artificial. Para estudiantes que ya desarrollan con soltura en lenguajes como Python o Java, Ray les permite distribuir su código en un clúster con mucha facilidad sin tener que aprender todo un paradigma nuevo como el de Spark.

4.4. Orquestación de Flujos de Trabajo

Cuando se tienen todos estos servicios de internet corriendo, se necesita algo que los coordine y automatice. Ahí es donde entran las herramientas de orquestación de flujos de trabajo:

- **Apache Airflow:** Creado por Airbnb, es la herramienta fundamental en cualquier arquitectura de datos moderna. Permite programar, orquestar y monitorear flujos de trabajo de datos (pipelines) mediante código, definiendo qué servicio se ejecuta, cuándo y en qué orden.
- **Prefect:** Similar a Airflow, Prefect es una herramienta de orquestación de flujos de trabajo que se centra en la simplicidad y la facilidad de uso. Ofrece una interfaz intuitiva para definir y ejecutar tareas, con características avanzadas para el manejo de errores y la gestión de dependencias.

Referencias

- [1] Amazon Web Services. «Arquitectura de los datos - AWS Marco de adopción de la nube: perspectiva de plataforma.» Guía prescriptiva sobre el diseño de arquitecturas de datos modernas y el desacomplamiento de cómputo y almacenamiento. (2024), dirección: https://docs.aws.amazon.com/es_es/prescriptive-guidance/latest/aws-caf-platform-perspective/data-arch.html.
- [2] Databricks. «¿Qué es la arquitectura de datos?» Conceptos fundamentales sobre el diseño, principios y modernización hacia el Data Lakehouse. (2024), dirección: <https://www.databricks.com/es/blog/what-is-data-architecture>.
- [3] IBM. «¿Qué es la arquitectura de datos?» Definición empresarial de componentes integrales, gestión y políticas de datos para inteligencia artificial. (2024), dirección: <https://www.ibm.com/mx-es/think/topics/data-architecture>.
- [4] BigBayData. «¿Qué es Hadoop y para qué sirve?» Conceptos fundamentales del ecosistema Hadoop para procesamiento de Big Data. (2024), dirección: <https://www.bigbaydata.com/hadoop-que-es/>.
- [5] AltexSoft. «Hadoop Pros and Cons: A Look at the Big Data Framework.» Análisis técnico de las ventajas y limitaciones de la infraestructura Hadoop. (2024), dirección: <https://www.altexsoft.com/blog/hadoop-pros-cons/>.
- [6] Apache Software Foundation, *HDFS Architecture Guide*, Documentación oficial sobre el diseño arquitectónico del Hadoop Distributed File System., 2024. dirección: <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>.
- [7] IONOS Digital Guide. «Apache Hadoop: El framework para Big Data.» Guía sobre la implementación y funcionamiento de Hadoop en servidores. (2024), dirección: <https://www.ionos.mx/digitalguide/servidores/know-how/apache-hadoop-el-framework-para-big-data/>.
- [8] Google Cloud, *Principales de IAM en Dataproc*, Documentación oficial sobre cuentas de servicio, roles y seguridad en clústeres administrados., 2024. dirección: <https://docs.cloud.google.com/dataproc/docs/concepts/iam/dataproc-principals?hl=es-419>.
- [9] NVIDIA. «Introduction to Spark Processing.» Conceptos básicos sobre el motor analítico de procesamiento en memoria de Apache Spark. (2024), dirección: <https://www.nvidia.com/es-la/ai-data-science/spark-ebook/introduction-spark-processing/>.
- [10] R. Bhatia. «Apache Spark Explained: Architecture.» Explicación detallada de la arquitectura interna de Apache Spark, incluyendo Drivers y Executors. (2024), dirección: <https://rockybhatia.substack.com/p/apache-spark-explained-architecture>.